

CampusHub SOLID 检查清单

1. 检查目的与范围

本清单用于完成 Phase 3 中“AI 设计缺陷注入”实验，检查对象为 CampusHub 在 P0 核心功能范围内的详细设计，重点覆盖以下链路：

- 1. 用户注册、登录与资料维护。
- 2. 需求发布、浏览与详情查看。
- 3. 接单、确认、进行中、完成的订单流转。

本次检查以 AI 生成的初版类设计为输入，逐条对照 SOLID 原则进行审查，并给出人工修正方案。修正后的结构与类图保持一致。

2. AI 原始设计摘要

为了体现 AI 在详细设计中的能力边界，团队先让 AI 生成了一个初版设计。该初版设计具有以下典型特征：

- 1. 使用一个 CampusHubManager 大类统一处理注册、登录、发布需求、接单、评价、通知和统计。
- 2. 订单状态变更通过 OrderService 中的大量 if-else 和 switch 实现。
- 3. 新增需求分类、推荐规则或信用规则时，需要直接修改原有服务类。
- 4. 使用一个大而全的 CampusHubService 接口暴露全部系统能力。
- 5. 高层业务服务直接依赖数据库实现类和消息发送实现类。

该设计适合快速形成草稿，但在可维护性、可扩展性和可测试性方面存在明显缺陷，因此需要进行 SOLID 逐条修正。

3. 总体检查结论

检查项	结果
检查原则数量	5 项
AI 原始设计中发现的典型违规数	5 处
是否覆盖全部 SOLID 原则	是

检查项	结果
是否给出对应修正方案	是
是否已在修正版类图中落实	是

结论：AI 原始设计中共发现 5 类典型问题，分别对应 S、O、L、I、D 五条原则。经过人工修订后，相关问题均已在修正版设计中得到针对性处理。

4. SOLID 逐条检查

4.1 S - 单一职责原则

项目	内容
检查问题	有没有类承担了过多职责？
AI 设计是否违反	是
违规说明	CampusHubManager 同时负责用户、需求、订单、评价、通知和统计。任一业务需求变化都可能导致该类修改，说明职责边界过于集中。
造成影响	类体积持续膨胀，维护难度增加；测试时需要构造过多上下文；后续多人协作时容易产生冲突。
修正方案	将大类拆分为 RequestService、OrderService、ReviewService、NotificationGateway 等职责单一的服务，并将 UserAccount、ServiceRequest、ServiceOrder 等领域实体独立出来。
修正后体现	用户、需求、订单、评价、通知分别由独立实体或服务负责，类职责边界清晰。

4.2 O - 开闭原则

项目	内容
检查问题	新增需求类型、状态规则或推荐规则时是否需要修改原有代码？
AI 设计是否违反	是

项目	内容
违规说明	初版 RequestService 使用 switch 判断需求分类，OrderService 使用条件分支处理状态变更。每次新增业务类型或规则都需要修改既有逻辑。
造成影响	新增功能会反复改动旧代码；容易引入回归问题；扩展越多，条件分支越复杂。
修正方案	用 OrderState 封装订单状态流转规则；用 MatchingStrategy 和 CreditPolicy 抽象可替换规则，把“新增功能”转化为“新增实现类”。
修正后体现	增加新状态或新策略时，优先扩展新的状态类或策略类，而不是直接修改原服务核心代码。

4.3 L - 里氏替换原则

项目	内容
检查问题	子类是否可以无异常地替换父类使用？
AI 设计是否违反	是
违规说明	AI 初版把 PendingOrder、CompletedOrder、CancelledOrder 设计成 ServiceOrder 的子类，但部分子类会在 complete 或 cancel 方法中抛异常，说明子类不能等价替代父类。
造成影响	调用方无法仅依赖父类语义进行编程；继承关系表面成立，实际行为不一致；设计容易误导实现。
修正方案	放弃用继承表达订单状态，改为 ServiceOrder 持有 OrderState 对象，把状态差异移动到状态类中。
修正后体现	ServiceOrder 作为统一订单实体保持稳定，状态相关行为由 PendingState、AcceptedState、InProgressState、CompletedState、CancelledState 分别承担。

4.4 I - 接口隔离原则

项目	内容
检查问题	有没有接口过于臃肿，包含调用方不需要的方法？

项目	内容
AI 设计是否违反	是
违规说明	初版使用统一 CampusHubService 接口，既包含注册登录，也包含订单处理、消息发送、统计分析。不同模块被迫依赖自己并不需要的方法。
造成影响	接口职责过宽；实现类和调用方耦合过强；局部修改可能影响无关模块。
修正方案	按业务拆分接口，例如 RequestService、OrderService、ReviewService、NotificationGateway，调用方只依赖自己真正需要的能力。
修正后体现	发布需求的调用方只依赖 RequestService；评价流程只依赖 ReviewService；通知能力通过 NotificationGateway 单独抽象。

4.5 D - 依赖倒转原则

项目	内容
检查问题	高层模块是否直接依赖低层模块的具体实现？
AI 设计是否违反	是
违规说明	OrderService 直接依赖 MySqlOrderRepository 和 WebSocketNotificationSender，导致难以替换存储方式和通知方式。
造成影响	持久化或通知技术一旦变化，高层业务代码也要跟着修改；单元测试难以替换底层依赖；系统演进成本变高。
修正方案	让业务层依赖 OrderRepository、RequestRepository、ReviewRepository、NotificationGateway 等抽象接口，具体实现放在基础设施层注入。
修正后体现	应用服务只感知仓储和通知抽象，具体数据库和推送实现可以单独替换。

5. 修正后的核心设计落点

修正后的设计已经落实，关键变化如下：

- 1. 使用 UserAccount、UserProfile、ServiceRequest、ServiceOrder、Review、Notification 作为核心领域实体。

2. 使用 RequestService、OrderService、ReviewService 作为应用服务接口，避免大而全的统一服务。
3. 使用 OrderState 管理订单流转，避免在 OrderService 中堆积条件分支。
4. 使用 MatchingStrategy 和 CreditPolicy 作为可扩展规则接口。
5. 使用 Repository 接口和 NotificationGateway 抽象底层实现。

6. AI 初版设计与人工修订对比

6.1 AI 初版设计的优点

1. 生成速度快，能快速覆盖主要功能点。
2. 适合作为团队讨论的第一版草稿。
3. 对于没有设计基础的阶段，能帮助快速暴露核心业务对象。

6.2 AI 初版设计的主要缺陷

1. 过度集中职责，容易生成大类和大接口。
2. 偏向用条件分支实现流程，不主动抽象状态与策略。
3. 容易忽略“依赖抽象而不是依赖实现”的工程约束。
4. 给出的方案表面完整，但后续修改成本较高。

6.3 人工修订的核心价值

1. 根据 SOLID 原则重建职责边界。
2. 将扩展点前置到设计阶段，而不是等代码膨胀后再返工。
3. 让类图既能支撑实现，也能支撑验收和答辩说明。

7. 实验记录与最终结论

本次“AI 设计缺陷注入”实验中，AI 原始设计共发现 5 处典型的 SOLID 违规问题，分别对应单一职责、开闭、里氏替换、接口隔离、依赖倒转五条原则。团队没有直接接受 AI 初稿，而是通过人工审查逐条定位问题，并在修正版设计中完成如下修正：

1. 将大类拆分为多个实体和服务接口。
2. 将订单状态差异从继承结构改为状态对象。
3. 将规则变化从条件分支改为策略扩展。
4. 将统一大接口拆分为多个小接口。
5. 将高层服务从具体数据库和通知实现中解耦。

最终，修正版设计满足了本阶段“记录 AI 原始设计中有多少处违反 SOLID 原则、修正了哪些”的任务要求，也为后续 API 设计和代码实现提供了稳定的详细设计基础。